PROJET · THÉORIE DES TESTS

TSE · MSc A1 EMDS (magistère)
Université Toulouse Capitole · 2026–2027

Dormir, jouer… gagner ?



Quels facteurs sont liés à la performance des joueurs d'esport ?

Trinôme 11

Joseph BERTIN · Thibault GUIHARD · Adam CHEBREK

Contexte, données et problématique

250 000 joueurs d'esport dans la base

150 tirés au hasard parmi 232 793 complets

12 variables : 9 quanti., 3 quali.

Source : Kaggle, « E-Sports Biometrics & Performance » (S. Chhetri), données partiellement simulées

Facteurs étudiés

Profil	âge, genre de jeu
Hygiène de vie	sommeil, jeu, boissons
Écran	luminosité, filtre
Physiologie	fréq. cardiaque, VFC

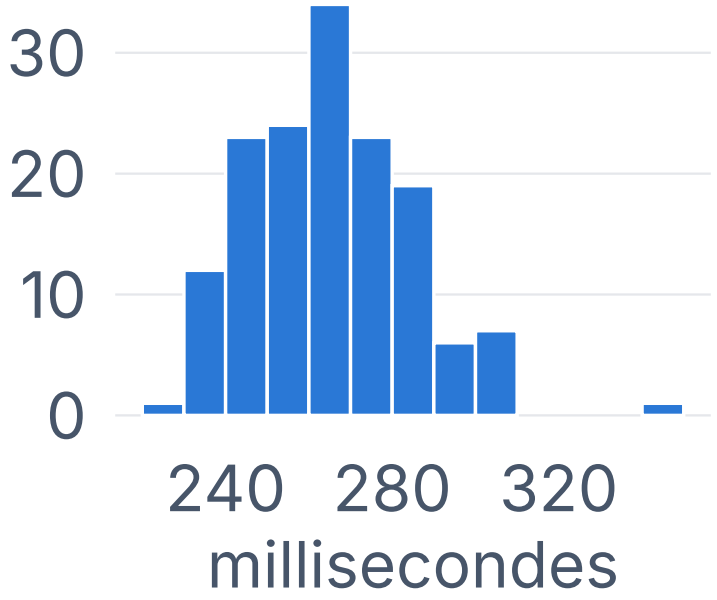
↓

Performance
temps de réaction · APM · issue du match

Problématique **Quels facteurs sont liés à la performance des joueurs d'esport ?**

Descriptif · les indicateurs de performance

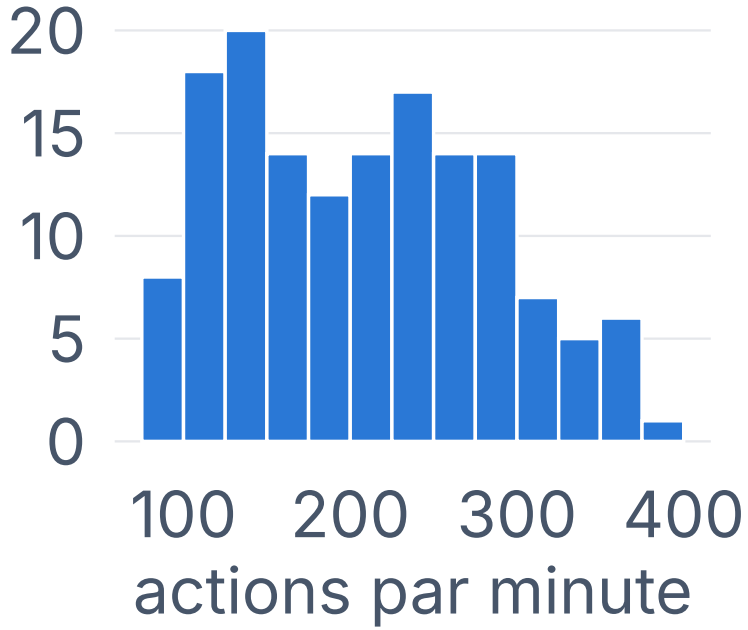
Temps de réaction



moy. 265,3 ms · σ 19,5

symétrique, 222 → 344

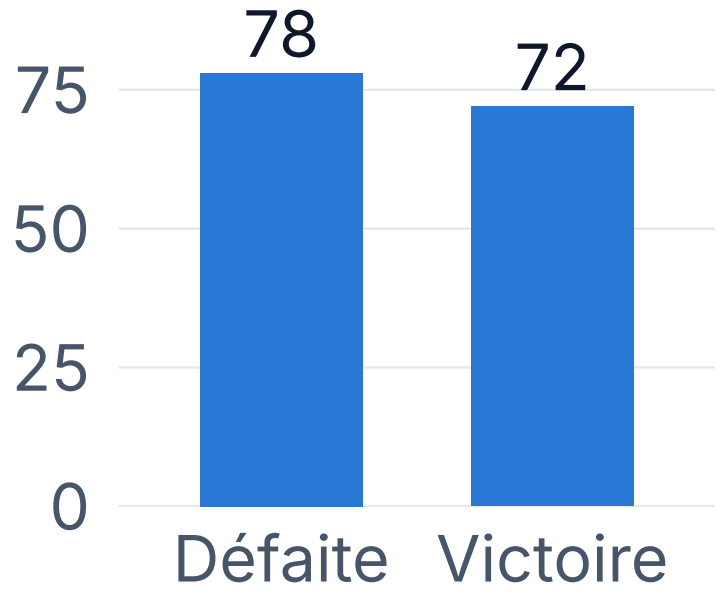
APM



méd. 207,5 · σ 78

très dispersé, 75 → 400

Issue du match



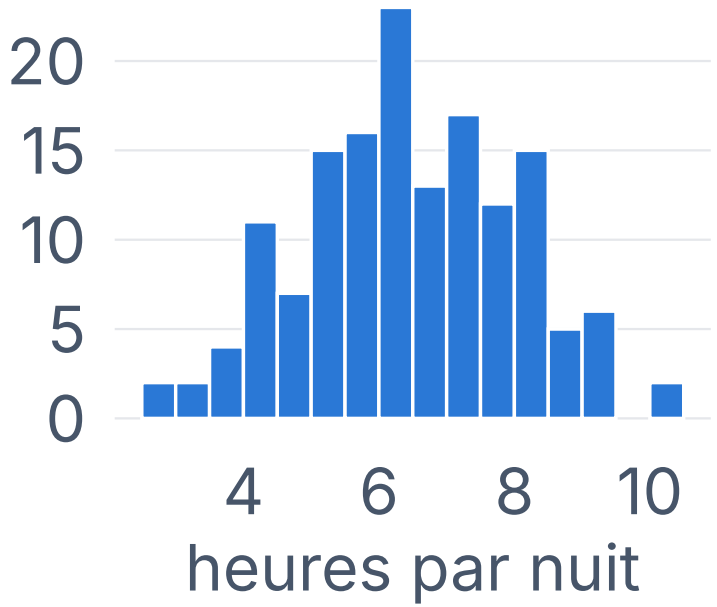
52 % de défaites

48 % de victoires

À retenir : réaction ≈ symétrique · APM très hétérogène (effet du genre ?) · issues équilibrées

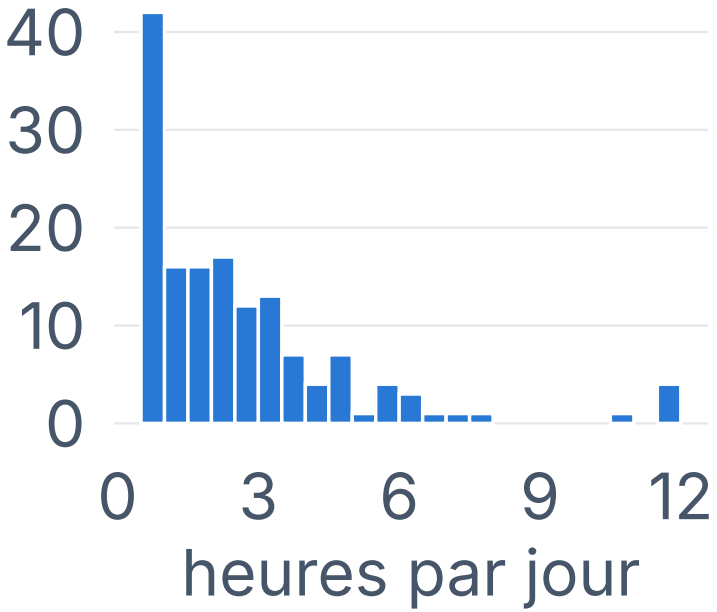
Descriptif · l'hygiène de vie

Sommeil



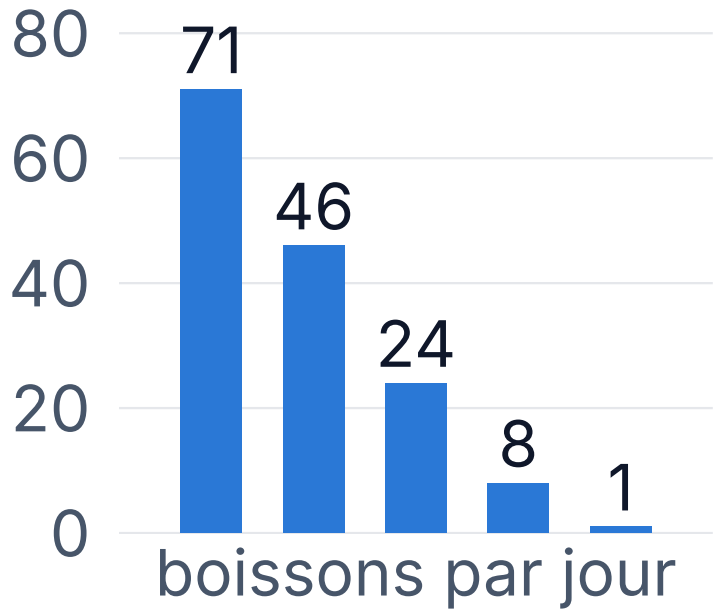
moy. 6,5 h · σ 1,6
38 % dorment < 6 h

Temps de jeu



méd. 2,1 h · moy. 2,6 h
asymétrique, 27 à 0,5 h

Boissons énergisantes

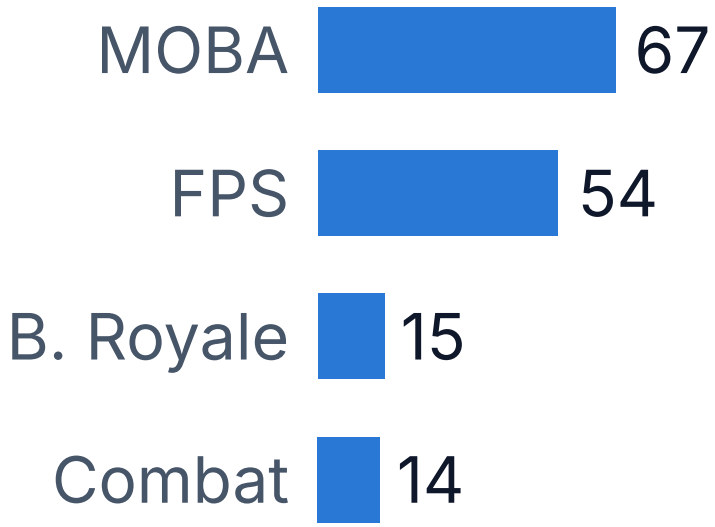


47 % n'en boivent pas
discrète, de 0 à 4

Temps de jeu découpé en 3 classes : ≤ 1 h (42) · 1–3 h (61) · > 3 h (47)

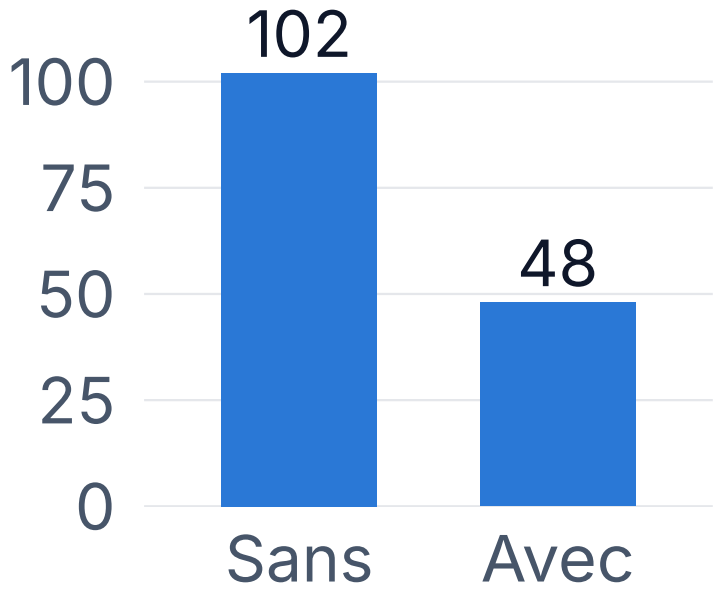
Descriptif · profil, écran et physiologie

Genre de jeu



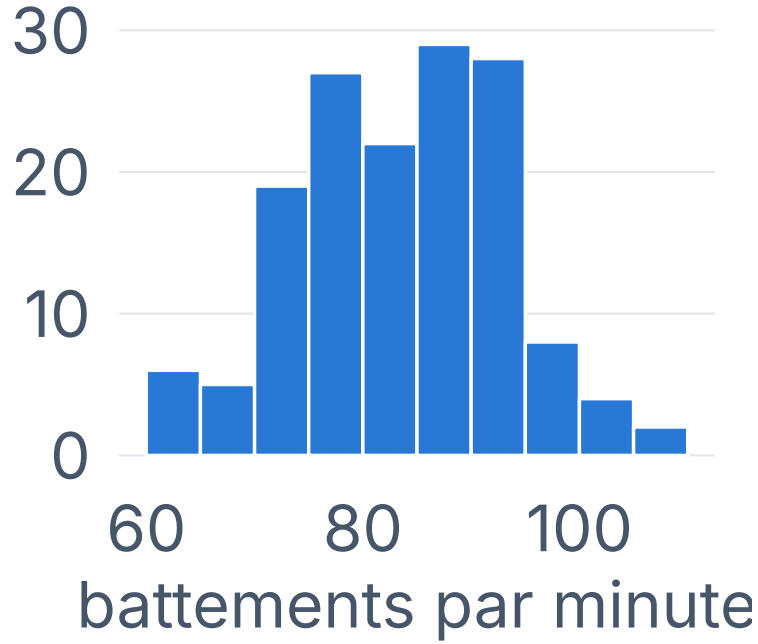
81 % MOBA ou FPS
B. Royale, Combat < 30

Filtre lumière bleue



32 % l'utilisent
68 % sans filtre

Fréquence cardiaque

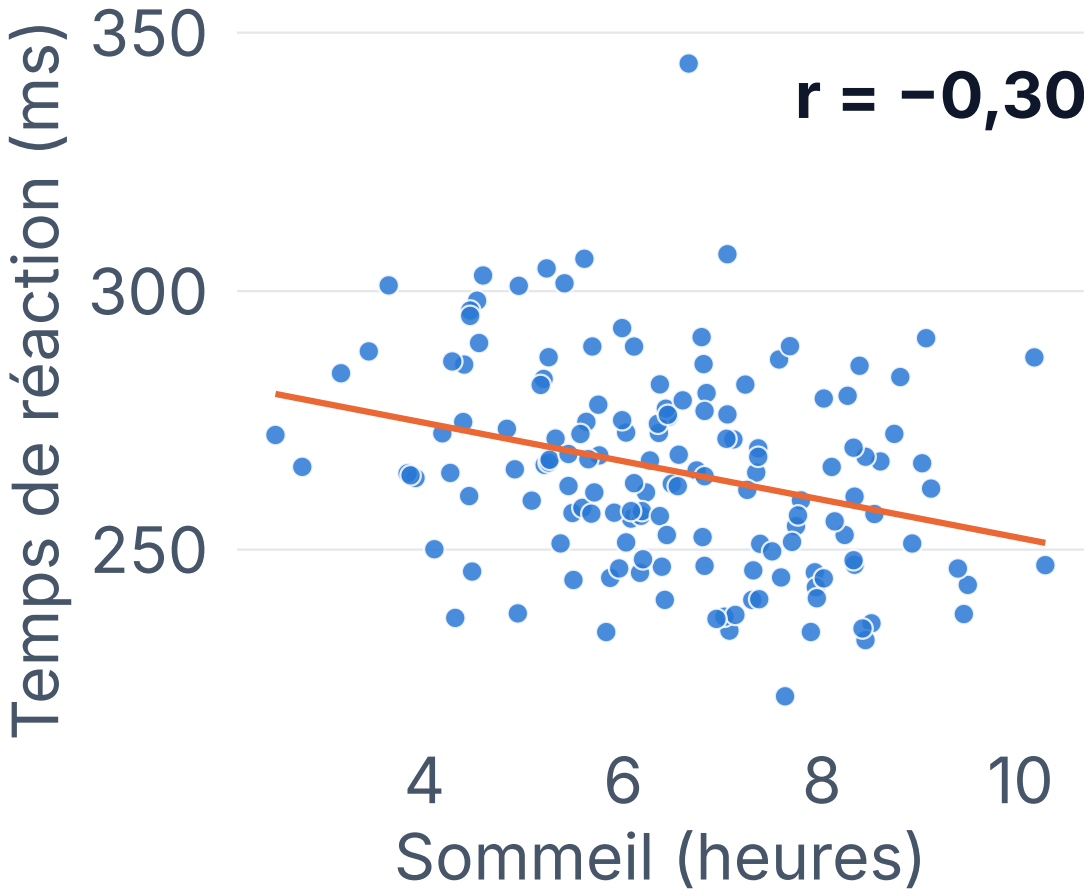


moy. 84 bpm · σ 10
de 60 à 110 bpm

Aussi : âge 16–33 ans (méd. 23) · VFC méd. 88 (16 au plafond 100)

1 Sommeil et temps de réaction

Pearson



H_0 ρ = 0 (pas de liaison linéaire)

H_1 ρ ≠ 0

- 2 variables quantitatives continues
- n = 150 ≥ 30 : test asymptotique, sans hypothèse de normalité

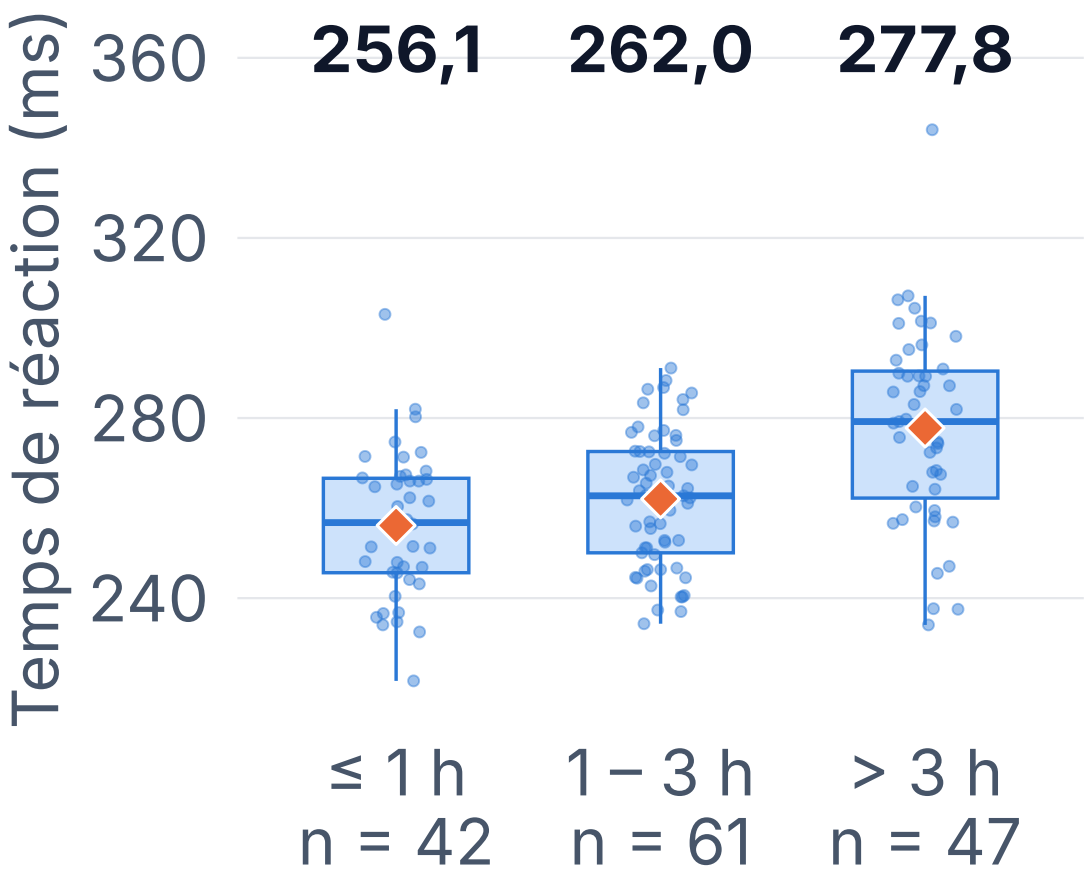
r = −0,30 · pente ≈ −3,7 ms/h

p-valeur **$2{,}4 \cdot 10^{-4}$** H_0 rejetée

Conclusion : liaison linéaire négative, modérée. Plus de sommeil va de pair avec une réaction plus rapide.

2 Temps de jeu et temps de réaction

ANOVA de Welch



H_0 $\mu_1 = \mu_2 = \mu_3$

H_1 au moins 2 moyennes diffèrent

- 3 groupes indépendants
- $n_k \geq 30$ $(42 \cdot 61 \cdot 47)$: asymptotique
- variances non supposées égales

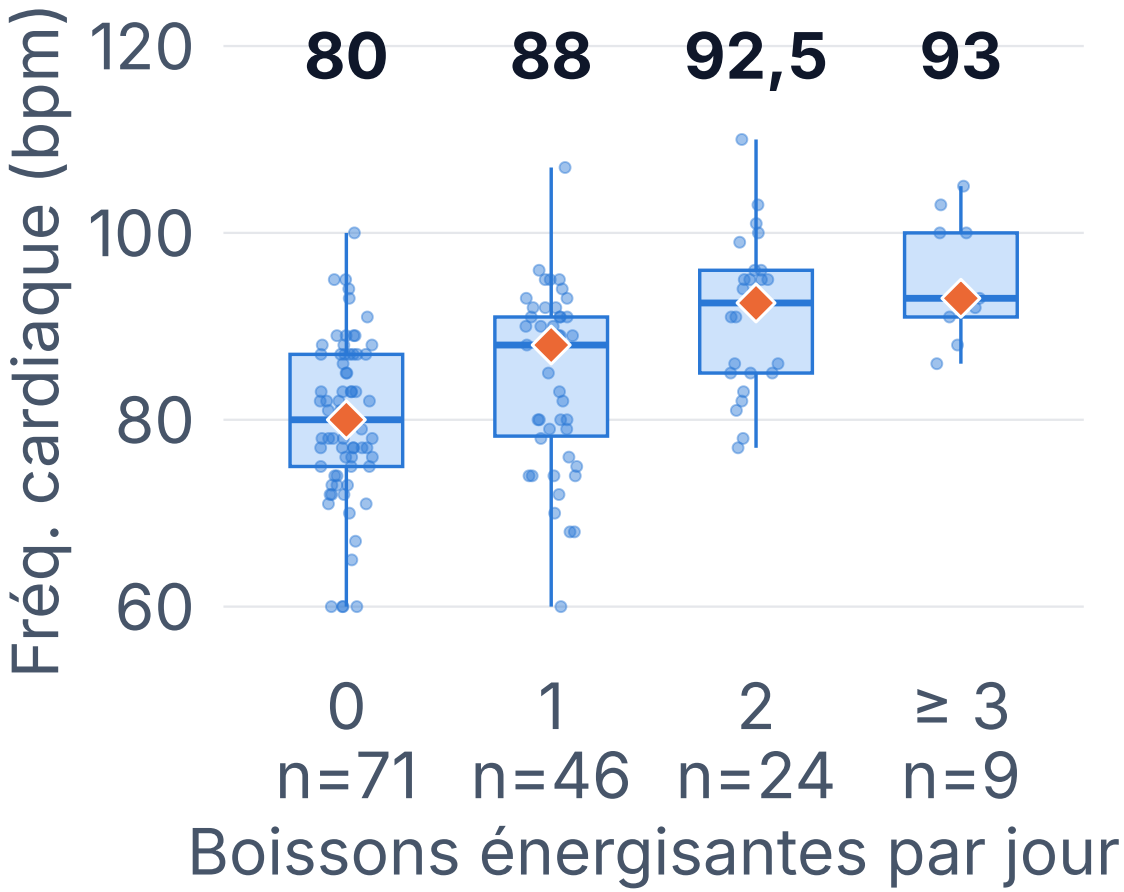
Bonferroni : seule la classe > 3 h diffère des deux autres

p-valeur $\mathbf{2{,}7 \cdot 10^{-6}}$ **H_0 rejetée**

Conclusion : au-delà de 3 h de jeu, réaction plus lente de 16 à 22 ms en moyenne (◆ moyenne).

3 Boissons énergisantes et rythme cardiaque

Spearman



H_0 $\rho_S = 0$ (pas de liaison monotone)

H_1 $\rho_S \neq 0$

- boissons : variable ordinale (0 à 4)
- nombreux ex-æquo → rangs
- n = 150 : test asymptotique

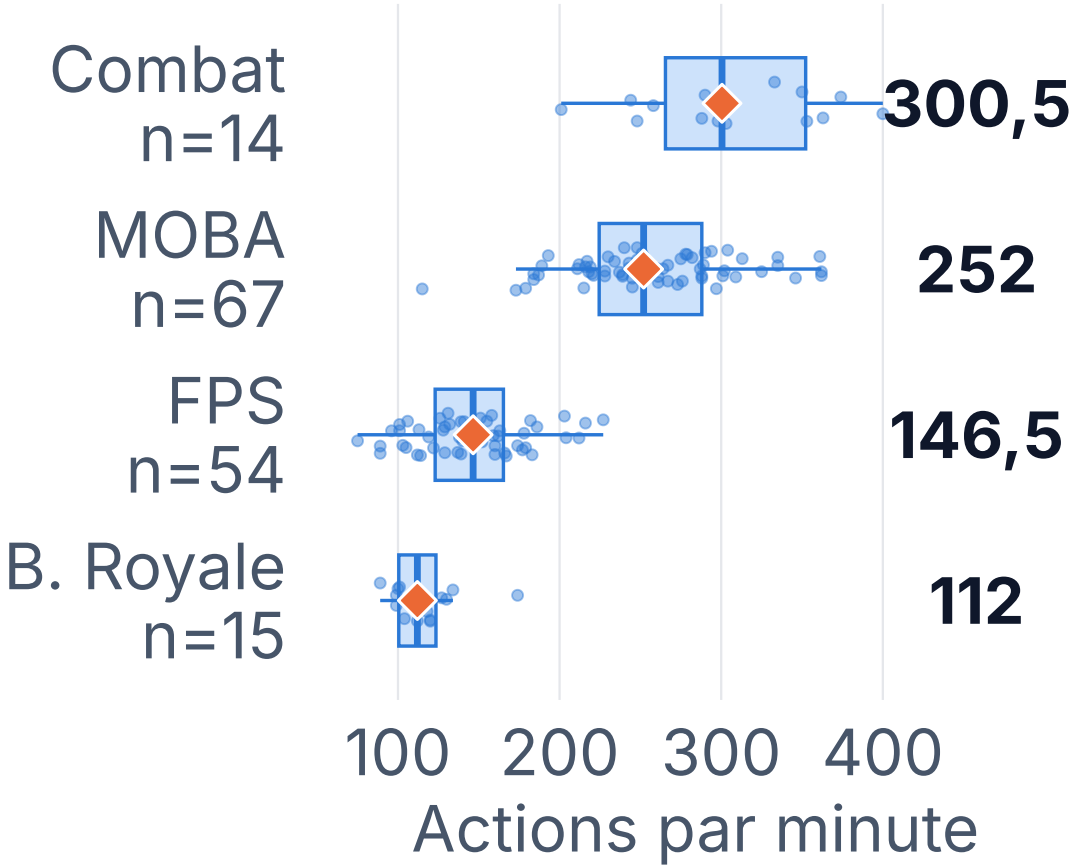
$\rho_S = 0{,}48$ · médianes ◆ 80 → 93 bpm

p-valeur **$4{,}1 \cdot 10^{-10}$** H_0 rejetée

Conclusion : liaison croissante. Plus de boissons énergisantes, plus le cœur bat vite (+13 bpm en médiane).

4 Genre de jeu et APM

Kruskal-Wallis



H_0 même loi d'APM dans les 4 genres

H_1 au moins un genre diffère

- 4 groupes indépendants
- B. Royale (15), Combat (14) < 30 → test non paramétrique

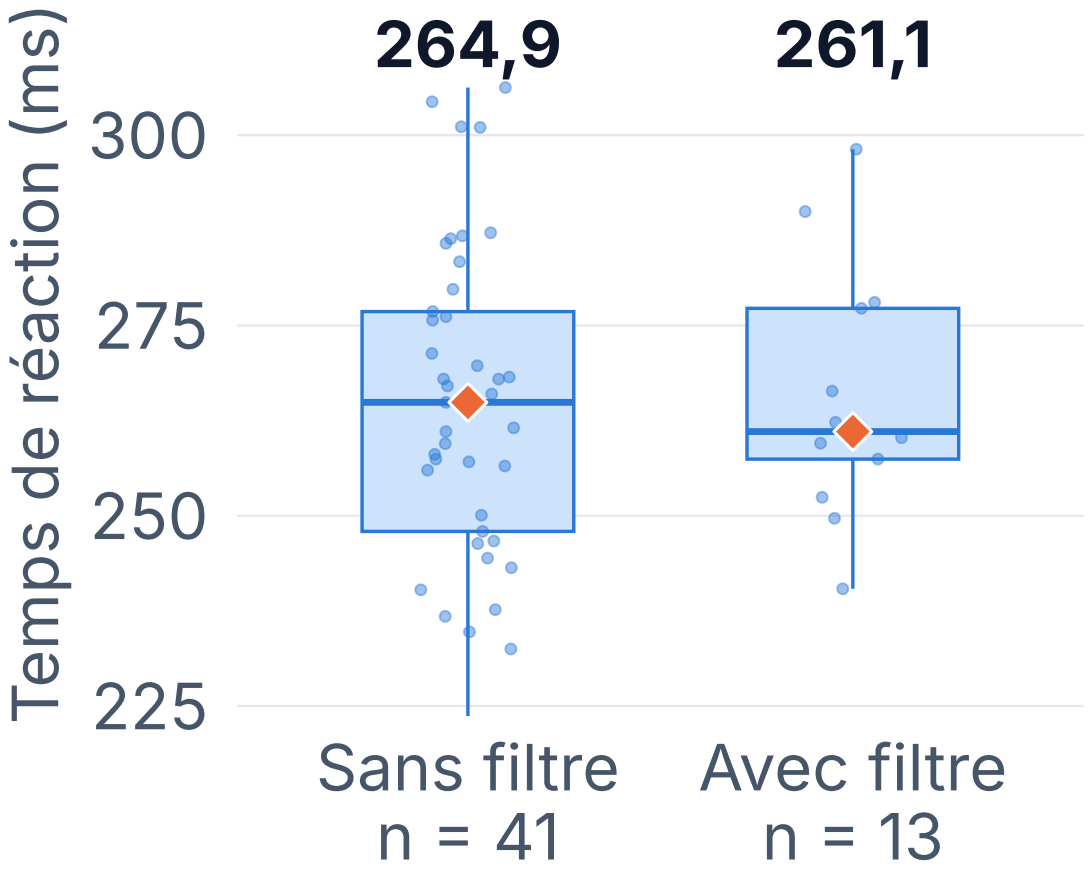
Wilcoxon 2 à 2 (Bonferroni) : les 6 paires diffèrent

p-valeur $\mathbf{3{,}7 \cdot 10^{-23}}$ **H_0 rejetée**

Conclusion : l'APM dépend fortement du genre. Combat et MOBA demandent 1,7 à 2,7 fois plus d'actions que FPS et B. Royale.

5 Filtre lumière bleue et réaction (FPS)

Mann-Whitney



Sous-échantillon : 54 joueurs de FPS, genre où les réflexes priment

H_0 même loi du temps de réaction avec ou sans filtre

H_1 lois décalées

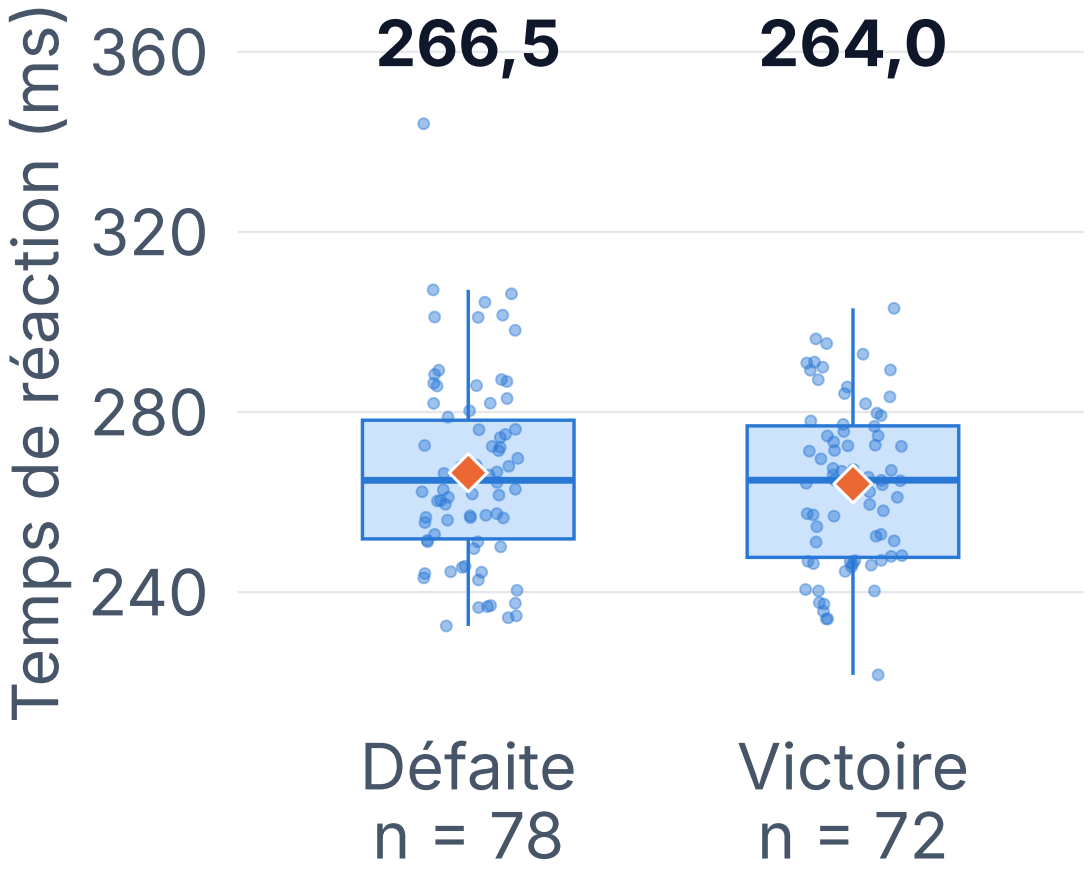
- 2 groupes indépendants
- avec filtre : n = 13 < 30 → test de Wilcoxon-Mann-Whitney

p-valeur **0,84** **H_0 non rejetée**

Conclusion : chez les joueurs de FPS, le filtre n'est pas lié au temps de réaction (médianes ◆ 265 et 261 ms).

6 Temps de réaction et issue du match

Student-Welch



H_0 $\mu_D = \mu_V$ (D : défaite, V : victoire)

H_1 $\mu_D \neq \mu_V$

- 2 groupes indépendants
- n = 78 et 72 ≥ 30 : test asymptotique
- variances non supposées égales

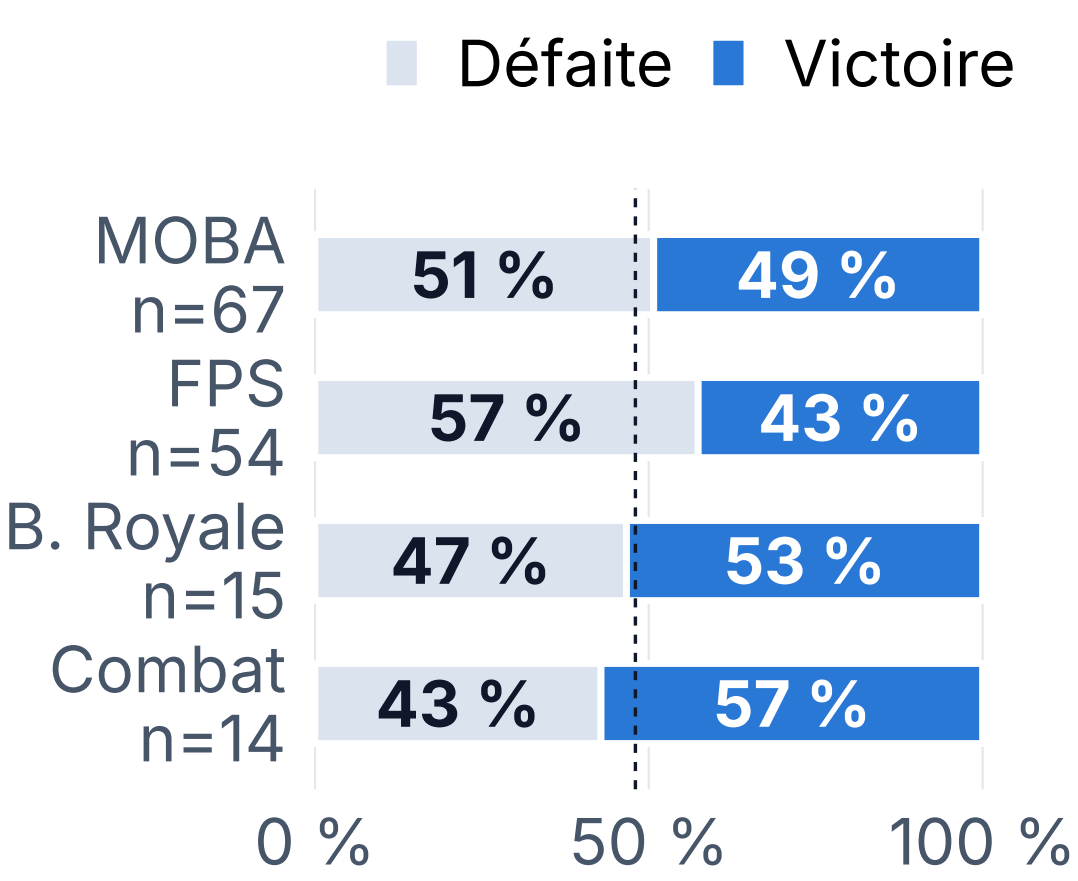
Moyennes ◆ : 266,5 et 264,0 ms, écart de 2,5 ms

p-valeur **0,44** **H_0 non rejetée**

Conclusion : gagnants et perdants réagissent aussi vite en moyenne. La réaction n'explique pas la victoire.

7 Genre de jeu et issue du match

χ^2 d'indépendance



H_0 genre et issue indépendants

H_1 genre et issue liés

- observations indépendantes
- effectifs théoriques ≥ 5 (min. 6,7)

Victoires : de 43 % (FPS) à 57 % (Combat)

pointillés = taux global de 48 %

p-valeur **0,73** **H_0 non rejetée**

Conclusion : le taux de victoire ne dépend pas du genre de jeu : il reste proche de 50 %.

Autres liaisons testées

TR = temps de réaction

Variable 1	Variable 2	Test	Coef.	p-valeur	Liaison (5 %)
Âge	TR	Pearson	0,40	$3 \cdot 10^{-7}$	● positive
VFC	TR	Pearson	−0,34	$2 \cdot 10^{-5}$	● négative
Sommeil	VFC	Pearson	0,37	$3 \cdot 10^{-6}$	● positive
Temps de jeu	VFC	Spearman	−0,27	$8 \cdot 10^{-4}$	● négative
FC	APM	Pearson	0,35	$1 \cdot 10^{-5}$	● positive
Boissons	TR	Spearman	0,03	0,72	aucune
Luminosité	TR	Pearson	−0,01	0,87	aucune
Genre	TR	Kruskal-Wallis	—	0,67	aucune
APM	Issue du match	Welch	—	0,84	aucune
Filtre	Issue du match	χ^2	—	0,87	aucune

Conclusion · ce qui est lié à la performance

Temps de réaction	**sommeil** (–), **sessions longues** (+), âge (+), VFC (–)
APM	**genre de jeu** (effet très fort), fréquence cardiaque (+)
Fréq. cardiaque	**boissons énergisantes** (+), sans effet sur la réaction
Issue du match	aucun facteur mesuré : ni réaction, ni genre, ni écran

Limites

- données partiellement simulées
- 17 tests : risque de faux positifs
- corrélation ≠ causalité
- petits groupes (FPS avec filtre : 13)

Réponse : les réflexes se jouent aussi hors de l'écran (sommeil, durée de jeu) ; la victoire dépend d'autres facteurs.

Annexe · code R utilisé (analyse_projet.R)

A1

```r
################################################################################
# Projet Théorie des tests — TSE, MSc A1 EMDS (magistère), 2026-2027
# Trinôme 11 : BERTIN Joseph · GUIHARD Thibault · CHEBREK Adam
# Données : Kaggle « E-Sports Biometrics & Performance » (S. Chhetri)
# Échantillon : 150 joueurs tirés au hasard parmi les 232 793 complets
# Problématique : quels facteurs sont liés à la performance en esport ?
# Seuil de tous les tests : alpha = 5 %
################################################################################

## 1. Données ------------------------------------------------------------------
d <- read.csv2("echantillon_150_complet.csv", stringsAsFactors = TRUE)
d$Game_Genre <- factor(d$Game_Genre,
                       levels = c("MOBA", "FPS", "Battle Royale", "Fighting"))
d$Match_Outcome <- factor(d$Match_Outcome, levels = c("Loss", "Win"))
str(d)
colSums(is.na(d))                 # aucune valeur manquante : 150 complets

# Variable qualitative ordinale obtenue en discrétisant le temps de jeu
d$Classe_Jeu <- cut(d$PlayTime_Hours, breaks = c(0, 1, 3, Inf),
                    labels = c("<= 1 h", "1-3 h", "> 3 h"))
table(d$Classe_Jeu)               # 42 / 61 / 47 : tous les effectifs >= 30

## 2. Étude descriptive univariée ----------------------------------------------
quanti <- c("Reaction_Time_ms", "APM", "Sleep_Hours", "PlayTime_Hours",
            "Energy_Drinks", "Heart_Rate_BPM", "HRV_Score", "Age",
            "Screen_Brightness")
resume <- function(x) c(moy = mean(x), et = sd(x), min = min(x),
                        q1 = unname(quantile(x, 0.25)), med = median(x),
                        q3 = unname(quantile(x, 0.75)), max = max(x))
round(t(sapply(d[quanti], resume)), 2)

quali <- c("Game_Genre", "Blue_Light_Filter", "Match_Outcome",
           "Energy_Drinks")
for (v in quali) {
  n <- table(d[[v]])
  print(cbind(n = n, pct = round(100 * prop.table(n), 1)))
}
sum(d$PlayTime_Hours == 0.5)      # joueurs au plancher de 0,5 h de jeu
sum(d$HRV_Score == 100)           # joueurs au plafond de 100 (VFC)
mean(d$Sleep_Hours < 6)           # part des joueurs dormant moins de 6 h

## 3. Les 7 tests de liaison obligatoires --------------------------------------

# Test 1 — Pearson asymptotique (n = 150) : sommeil ~ temps de réaction
# H0 : rho = 0   contre   H1 : rho != 0
cor.test(d$Sleep_Hours, d$Reaction_Time_ms, method = "pearson")
coef(lm(Reaction_Time_ms ~ Sleep_Hours, data = d))  # pente : ms par heure

# Test 2 — ANOVA de Welch asymptotique : temps de réaction ~ durée de jeu
# H0 : mu1 = mu2 = mu3   contre   H1 : au moins deux moyennes diffèrent
resume_groupe <- function(x)
  round(c(n = length(x), moy = mean(x), et = sd(x)), 1)
aggregate(Reaction_Time_ms ~ Classe_Jeu, data = d, FUN = resume_groupe)
oneway.test(Reaction_Time_ms ~ Classe_Jeu, data = d, var.equal = FALSE)
pairwise.t.test(d$Reaction_Time_ms, d$Classe_Jeu, pool.sd = FALSE,
                p.adjust.method = "bonferroni")   # comparaisons multiples

# Test 3 — Spearman : boissons énergisantes ~ fréquence cardiaque
# H0 : rho_S = 0   contre   H1 : rho_S != 0
# (nombreux ex-aequo -> approximation asymptotique, exact = FALSE)
cor.test(d$Energy_Drinks, d$Heart_Rate_BPM, method = "spearman",
         exact = FALSE)
tapply(d$Heart_Rate_BPM, pmin(d$Energy_Drinks, 3), median)
```

Annexe · code R utilisé (analyse_projet.R)

A2

```r
# Test 4 — Kruskal-Wallis : APM ~ genre (Battle Royale 15, Fighting 14)
# H0 : même loi de l'APM dans les 4 genres   contre   H1 : au moins une
# diffère
tapply(d$APM, d$Game_Genre, median)
kruskal.test(APM ~ Game_Genre, data = d)
kruskal.test(APM ~ Game_Genre, data = d)$p.value
pairwise.wilcox.test(d$APM, d$Game_Genre, p.adjust.method = "bonferroni",
                     exact = FALSE)              # comparaisons multiples

# Test 5 — Wilcoxon-Mann-Whitney : filtre lumière bleue ~ temps de
# réaction, sous-échantillon des joueurs de FPS (avec filtre : 13 < 30)
# H0 : même loi du temps de réaction avec et sans filtre
fps <- subset(d, Game_Genre == "FPS")
table(fps$Blue_Light_Filter)
tapply(fps$Reaction_Time_ms, fps$Blue_Light_Filter, median)
wilcox.test(Reaction_Time_ms ~ Blue_Light_Filter, data = fps)

# Test 6 — Student, version Welch asymptotique : temps de réaction ~ issue
# H0 : mu_défaite = mu_victoire   contre   H1 : mu_défaite != mu_victoire
aggregate(Reaction_Time_ms ~ Match_Outcome, data = d, FUN = resume_groupe)
tw <- t.test(Reaction_Time_ms ~ Match_Outcome, data = d, var.equal = FALSE)
tw
2 * pnorm(-abs(tw$statistic))      # p-valeur asymptotique (loi N(0, 1))

# Test 7 — Khi-deux d'indépendance : genre de jeu x issue du match
# H0 : les deux variables sont indépendantes
tab <- table(d$Game_Genre, d$Match_Outcome)
chisq.test(tab)$expected           # effectifs théoriques tous >= 5
chisq.test(tab)
round(100 * prop.table(tab, margin = 1), 1)
```

```r
## 4. Autres tests de liaison (tableau de synthèse) -------------------------
pearson  <- function(x, y) cor.test(d[[x]], d[[y]])
spearman <- function(x, y) cor.test(d[[x]], d[[y]], method = "spearman",
                                    exact = FALSE)
autres <- list(
  "Age - TR"           = pearson("Age", "Reaction_Time_ms"),
  "VFC - TR"           = pearson("HRV_Score", "Reaction_Time_ms"),
  "Sommeil - VFC"      = pearson("Sleep_Hours", "HRV_Score"),
  "Temps de jeu - VFC" = spearman("PlayTime_Hours", "HRV_Score"),
  "FC - APM"           = pearson("Heart_Rate_BPM", "APM"),
  "Boissons - TR"      = spearman("Energy_Drinks", "Reaction_Time_ms"),
  "Luminosite - TR"    = pearson("Screen_Brightness", "Reaction_Time_ms"),
  "Genre - TR"         = kruskal.test(Reaction_Time_ms ~ Game_Genre, d),
  "APM - Issue"        = t.test(APM ~ Match_Outcome, data = d),
  "Filtre - Issue"     = chisq.test(table(d$Blue_Light_Filter,
                                          d$Match_Outcome))
)
coef_test <- function(t)
  unname(if (length(t$estimate) == 1) t$estimate else t$statistic)
p_val <- sapply(autres, `[[`, "p.value")
data.frame(stat = round(sapply(autres, coef_test), 3),
           p_valeur = signif(p_val, 2),
           conclusion = ifelse(p_val < 0.05, "liaison significative",
                               "pas de liaison"))
```

Annexe · code R utilisé (analyse_projet.R)

A3

```r
## 5. Graphiques (exportés en SVG pour le support) --------------------------
library(ggplot2)
set.seed(11)                        # position aléatoire des points (jitter)
options(OutDec = ",")               # virgule décimale sur les graphiques
dir.create("figures", showWarnings = FALSE)
bleu <- "#2a78d6"; bleu_clair <- "#cde2fb"; orange <- "#eb6834"
encre <- "#0f172a"; gris <- "#475569"; grille <- "#e5e7eb"
theme_set(theme_minimal(base_size = 24, base_family = "Inter") +
  theme(panel.grid.minor = element_blank(),
        panel.grid.major.x = element_blank(),
        panel.grid.major.y = element_line(colour = grille, linewidth = 0.4),
        axis.text = element_text(colour = gris, size = 24),
        axis.title = element_text(colour = gris, size = 24),
        legend.text = element_text(size = 24),
        plot.margin = margin(12, 14, 4, 4)))
sauve <- function(p, nom, l, h)    # l et h en pixels (96 px = 1 pouce)
  ggsave(file.path("figures", paste0(nom, ".svg")), p,
         width = l / 96, height = h / 96)
txt <- function(...)
  list(size = 24 / .pt, family = "Inter", colour = encre, ...)
virg <- function(x, chiffres) formatC(x, format = "f", digits = chiffres,
                                      decimal.mark = ",")
virg_med <- function(x) ifelse(x == round(x), virg(x, 0), virg(x, 1))
```

```r
# Boîtes à moustaches + points. Losange orange = moyenne ou médiane (f) ;
# valeur de f écrite en haut de chaque groupe (ordonnée h)
boites <- function(df, x, y, lab_y, f, h, ylim, horiz = FALSE) {
  n <- table(df[[x]])
  sep <- if (length(n) > 3) "\nn=" else "\nn = "   # étiquettes compactes
  df$g <- factor(df[[x]], labels = paste0(names(n), sep, n))
  s <- aggregate(df[[y]], list(g = df$g), f)
  ggplot(df, aes(g, .data[[y]])) +
    geom_boxplot(fill = bleu_clair, colour = bleu, width = 0.55,
                 outlier.shape = NA, linewidth = 0.6) +
    geom_jitter(width = 0.13, height = 0, colour = bleu, alpha = 0.45,
                size = 1.6) +
    geom_point(data = s, aes(g, x), shape = 23, size = 4.8, fill = orange,
               colour = "white", stroke = 0.8) +
    do.call(annotate, c(list("text", x = seq_len(nrow(s)), y = h,
                             label = if (identical(f, mean))
                               virg(s$x, 1) else virg_med(s$x),
                             fontface = "bold"), txt())) +
    (if (horiz) coord_flip(ylim = ylim) else coord_cartesian(ylim = ylim)) +
    labs(x = NULL, y = lab_y)
}
histo <- function(v, pas, lab) ggplot(d, aes(.data[[v]])) +
  geom_histogram(binwidth = pas, boundary = 0, fill = bleu,
                 colour = "white", linewidth = 0.6) +
  labs(x = lab, y = NULL)
barres <- function(v, lab, noms) ggplot(d, aes(.data[[v]])) +
  geom_bar(fill = bleu, width = 0.6) +
  do.call(geom_text, c(list(stat = "count", vjust = -0.35,
                            aes(label = after_stat(count))), txt())) +
  scale_x_discrete(labels = noms) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.22))) +
  labs(x = lab, y = NULL)
```

Annexe · code R utilisé (analyse_projet.R)


```r
# Étude descriptive univariée
L <- 376; H <- 330
sauve(histo("Reaction_Time_ms", 10, "millisecondes"), "d_tr", L, H)
sauve(histo("APM", 25, "actions par minute"), "d_apm", L, H)
sauve(barres("Match_Outcome", NULL, c("Défaite", "Victoire")),
      "d_issue", L, H)
sauve(histo("Sleep_Hours", 0.5, "heures par nuit"), "d_sommeil", L, H)
sauve(histo("PlayTime_Hours", 0.5, "heures par jour") +
        scale_x_continuous(breaks = seq(0, 12, 3)), "d_jeu", L, H)
sauve(barres("Energy_Drinks", "boissons par jour", as.character(0:4)),
      "d_boissons", L, H)
genres <- c("MOBA", "FPS", "B. Royale", "Combat")
sauve(ggplot(d, aes(y = factor(Game_Genre, rev(levels(Game_Genre))))) +
        geom_bar(fill = bleu, width = 0.6) +
        do.call(geom_text, c(list(stat = "count", hjust = -0.25,
                                  aes(label = after_stat(count))), txt())) +
        scale_y_discrete(labels = rev(genres)) +
        scale_x_continuous(expand = expansion(mult = c(0, 0.32))) +
        labs(x = NULL, y = NULL) +
        theme(panel.grid.major.y = element_blank(),
              axis.text.x = element_blank()), "d_genre", L, H)
sauve(barres("Blue_Light_Filter", NULL, c("Sans", "Avec")),
      "d_filtre", L, H)
sauve(histo("Heart_Rate_BPM", 5, "battements par minute") +
        scale_x_continuous(breaks = seq(60, 110, 20)), "d_fc", L, H)
```

```r
# Test 1 : nuage de points + droite des moindres carrés
L <- 560; H <- 452
sauve(ggplot(d, aes(Sleep_Hours, Reaction_Time_ms)) +
        geom_point(shape = 21, fill = bleu, colour = "white", size = 3.2,
                   stroke = 0.6, alpha = 0.85) +
        geom_smooth(method = "lm", formula = y ~ x, se = FALSE,
                    colour = orange, linewidth = 1.1) +
        do.call(annotate, c(list("text", x = 9.2, y = 338,
                                 label = "r = -0,30",
                                 fontface = "bold"), txt())) +
        labs(x = "Sommeil (heures)", y = "Temps de réaction (ms)"),
      "t1_pearson", L, H)

# Test 2 : temps de réaction par classe de temps de jeu (moyennes)
d2 <- transform(d, Classe_Jeu = factor(Classe_Jeu,
                  labels = c("≤ 1 h", "1 – 3 h", "> 3 h")))
sauve(boites(d2, "Classe_Jeu", "Reaction_Time_ms", "Temps de réaction (ms)",
             mean, 362, c(220, 362)), "t2_anova", L, H)

# Test 3 : fréquence cardiaque par nombre de boissons (médianes ;
# 3 et 4 boissons regroupés sur le graphique uniquement)
d3 <- transform(d, Boissons = factor(pmin(Energy_Drinks, 3),
                  labels = c("0", "1", "2", "≥ 3")))
sauve(boites(d3, "Boissons", "Heart_Rate_BPM", "Fréq. cardiaque (bpm)",
             median, 119, c(58, 119)) +
        labs(x = "Boissons énergisantes par jour"), "t3_spearman", L, H)
```

Annexe · code R utilisé (analyse_projet.R)


```r
# Test 4 : APM par genre de jeu (boîtes horizontales, rangées par médiane)
d4 <- transform(d, Game_Genre = factor(Game_Genre, labels = genres))
d4$Game_Genre <- reorder(d4$Game_Genre, d4$APM, FUN = median)
sauve(boites(d4, "Game_Genre", "APM", "Actions par minute", median, 458,
             c(70, 498), horiz = TRUE) +
        scale_y_continuous(breaks = seq(100, 400, 100)) +
        theme(panel.grid.major.y = element_blank(),
              panel.grid.major.x = element_line(colour = grille,
                                                linewidth = 0.4)),
      "t4_kruskal", L, H)

# Test 5 : temps de réaction selon le filtre, joueurs de FPS (médianes)
d5 <- transform(fps, Blue_Light_Filter = factor(Blue_Light_Filter,
                  labels = c("Sans filtre", "Avec filtre")))
sauve(boites(d5, "Blue_Light_Filter", "Reaction_Time_ms",
             "Temps de réaction (ms)", median, 312, c(228, 312)),
      "t5_wilcoxon", L, H)

# Test 6 : temps de réaction selon l'issue du match (moyennes)
d6 <- transform(d, Match_Outcome = factor(Match_Outcome,
                  labels = c("Défaite", "Victoire")))
sauve(boites(d6, "Match_Outcome", "Reaction_Time_ms",
             "Temps de réaction (ms)", mean, 362, c(220, 362)),
      "t6_welch", L, H)
```

```r
# Test 7 : part de victoires par genre (barres empilées à 100 %)
p7 <- as.data.frame(prop.table(tab, 1))
names(p7) <- c("genre", "issue", "part")
p7$genre <- factor(p7$genre, levels = rev(levels(d$Game_Genre)),
                   labels = rev(paste0(genres, "\nn=", table(d$Game_Genre))))
p7$issue <- factor(p7$issue, labels = c("Défaite", "Victoire"))
sauve(ggplot(p7, aes(part, genre, fill = issue)) +
        geom_col(width = 0.56, colour = "white", linewidth = 1.2,
                 position = position_stack(reverse = TRUE)) +
        geom_text(aes(label = paste0(round(100 * part), " %"),
                      colour = issue), size = 24 / .pt, family = "Inter",
                  fontface = "bold",
                  position = position_stack(vjust = 0.5, reverse = TRUE)) +
        geom_vline(xintercept = mean(d$Match_Outcome == "Win"),
                   colour = encre, linewidth = 0.6, linetype = "22") +
        scale_fill_manual(values = c(Défaite = "#dbe3ee", Victoire = bleu)) +
        scale_colour_manual(values = c(Défaite = encre, Victoire = "white"),
                            guide = "none") +
        scale_x_continuous(labels = function(x) paste0(100 * x, " %"),
                           breaks = c(0, 0.5, 1),
                           expand = expansion(mult = c(0, 0.06))) +
        labs(x = NULL, y = NULL, fill = NULL) +
        theme(legend.position = "top", legend.justification = "left",
              plot.margin = margin(6, 34, 4, 4),
              panel.grid.major.y = element_blank(),
              panel.grid.major.x = element_line(colour = grille,
                                                linewidth = 0.4)),
      "t7_khi2", L, H)
```